

CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY

Faculty of Technology & Engineering

Devang Patel Institute of Advance Technology and Research (DEPSTAR)

CSUE301 – : Big Data Analytics

Practical File – ODD Semester 2026-27

StudentID: 24DIT066

Student Name: Sil Shah

Practical 3

Aim: To implement a **Word Count / Frequency Program using MapReduce** in Hadoop and analyze word frequencies in a text dataset stored in HDFS.

Problem definition:

A digital publishing company stores thousands of news articles and blog posts in the Hadoop Distributed File System (HDFS). The editorial team wants to identify the most frequently used words across their content to understand trending topics and optimize content recommendations.

As a Big Data Engineer, the task is to develop a MapReduce application that processes text data and generates word frequency statistics.

Objectives:

- Configure the Hadoop environment and HDFS.
- Create and upload a text dataset to HDFS.
- Implement a **Mapper** to generate (word, 1) key-value pairs.
- Implement a **Reducer** to calculate word frequencies.
- Execute the MapReduce job on Hadoop.
- Store the output in HDFS.
- Find the top 10 most frequently occurring words.
- Analyze the performance for different dataset sizes.

Observation / Output

Step 1: Check Hadoop Environment

```
hadoop@sil-shah-VMware-Virtual-Platform:~$ java -version
openjdk version "11.0.31" 2026-04-21
OpenJDK Runtime Environment (build 11.0.31+11-post-1ubuntu1-26.04.2-Ubuntu)
OpenJDK 64-Bit Server VM (build 11.0.31+11-post-1ubuntu1-26.04.2-Ubuntu, mixed mode, sharing)
hadoop@sil-shah-VMware-Virtual-Platform:~$ hadoop version
Hadoop 3.3.6
Source code repository https://github.com/apache/hadoop.git -r 1be70230728da9266a4f08195058f00fd012bf9c
Compiled by ubuntu on 2023-06-18T08:22Z
Compiled on platform linux-x86_64
Compiled with protoc 3.7.1
From source with checksum 5652179ad55f76cb287d9c633bb53bbd
This command was run using /usr/local/hadoop/share/hadoop/common/hadoop-common-3.3.6.jar
hadoop@sil-shah-VMware-Virtual-Platform:~$ hdfs dfs -ls /
ls: Call From sil-shah-VMware-Virtual-Platform/127.0.0.1 to localhost:9000 failed on connection exception: java.net.ConnectException: Connection refused; For more details see: http://wiki.apache.org/hadoop/ConnectionRefused
hadoop@sil-shah-VMware-Virtual-Platform:~$ jps
5815 Jps
hadoop@sil-shah-VMware-Virtual-Platform:~$
hadoop@sil-shah-VMware-Virtual-Platform:~$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [sil-shah-VMware-Virtual-Platform]
hadoop@sil-shah-VMware-Virtual-Platform:~$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
hadoop@sil-shah-VMware-Virtual-Platform:~$ jps
6658 ResourceManager
6036 NameNode
6791 NodeManager
7178 Jps
6426 SecondaryNameNode
6186 DataNode
hadoop@sil-shah-VMware-Virtual-Platform:~$ hdfs dfs -ls /
Found 2 items
drwxr-xr-x - hadoop supergroup 0 2026-08-06 20:48 /backup
drwxr-xr-x - hadoop supergroup 0 2026-07-18 19:18 /test
```

Step 2: Create the Input Dataset

```
hadoop@sil-shah-VMware-Virtual-Platform:~$ nano input.txt
hadoop@sil-shah-VMware-Virtual-Platform:~$ ls
Desktop Documents Downloads Music Pictures Public Templates Videos customer_sales.txt hadoop-3.3.6.tar.gz input.txt sales_data.txt snap
hadoop@sil-shah-VMware-Virtual-Platform:~$ cat input.txt
Hadoop is a big data technology.
Hadoop processes large data using distributed computing.
Big data helps organizations analyze large amounts of information.
Hadoop provides reliable and scalable data processing.
MapReduce is used to process big data efficiently.
Hadoop and MapReduce are important technologies for big data analytics.
hadoop@sil-shah-VMware-Virtual-Platform:~$
```

Step 3: Create HDFS Directory and Upload Dataset

```
hadoop@sil-shah-VMware-Virtual-Platform:~$ hdfs dfs -mkdir -p /wordcount/input
hadoop@sil-shah-VMware-Virtual-Platform:~$ hdfs dfs -ls /wordcount
Found 1 items
drwxr-xr-x - hadoop supergroup 0 2026-08-14 10:24 /wordcount/input
hadoop@sil-shah-VMware-Virtual-Platform:~$ hdfs dfs -put input.txt /wordcount/input/
hadoop@sil-shah-VMware-Virtual-Platform:~$ hdfs dfs -ls /wordcount/input
Found 1 items
-rw-r--r-- 1 hadoop supergroup 335 2026-08-14 10:25 /wordcount/input/input.txt
hadoop@sil-shah-VMware-Virtual-Platform:~$ hdfs dfs -cat /wordcount/input/input.txt
Hadoop is a big data technology.
Hadoop processes large data using distributed computing.
Big data helps organizations analyze large amounts of information.
Hadoop provides reliable and scalable data processing.
MapReduce is used to process big data efficiently.
Hadoop and MapReduce are important technologies for big data analytics.
hadoop@sil-shah-VMware-Virtual-Platform:~$
```

Step 4: Create the Java MapReduce Program

```
hadoop@sil-shah-VMware-Virtual-Platform:~$ nano WordCount.java
hadoop@sil-shah-VMware-Virtual-Platform:~$ ls
Desktop Documents Downloads Music Pictures Public Templates Videos WordCount.java customer_sales.txt hadoop-3.3.6.tar.gz input.txt sales_data.txt snap
hadoop@sil-shah-VMware-Virtual-Platform:~$ cat WordCount.java
import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

    public static class TokenizerMapper
        extends Mapper<Object, Text, Text, IntWritable> {

        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();

        public void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {

            String[] words = value.toString().split("\\s+");
            for (String word : words) {
                word = word.toLowerCase();
                context.write(new Text(word), one);
            }
        }
    }

    public static class Reducer
        extends Reducer<Text, IntWritable, Text, IntWritable> {

        private IntWritable sum = new IntWritable(0);

        public void reduce(Text key, Iterable<IntWritable> values, Context context)
            throws IOException, InterruptedException {
            for (IntWritable val : values) {
                sum.set(sum.get() + val.get());
            }
            context.write(key, sum);
        }
    }

    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = new Job(conf, "WordCount");
        job.setMapperClass(TokenizerMapper.class);
        job.setReducerClass(Reducer.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        job.waitForCompletion(true);
    }
}
```

Step 5: Compile the Java MapReduce Program

```
hadoop@sil-shah-VMware-Virtual-Platform:~$ mkdir -p classes
hadoop@sil-shah-VMware-Virtual-Platform:~$ javac -classpath "$(hadoop classpath)" -d classes WordCount.java
hadoop@sil-shah-VMware-Virtual-Platform:~$ find classes -type f
classes/WordCount.class
classes/WordCount$IntSumReducer.class
classes/WordCount$TokenizerMapper.class
hadoop@sil-shah-VMware-Virtual-Platform:~$ jar -cvf wordcount.jar -C classes/ .
added manifest
adding: WordCount.class(in = 1511)(out= 829)(deflated 45%)
adding: WordCount$IntSumReducer.class(in = 1657)(out= 700)(deflated 57%)
adding: WordCount$TokenizerMapper.class(in = 1865)(out= 818)(deflated 56%)
hadoop@sil-shah-VMware-Virtual-Platform:~$ ls -lh wordcount.jar
-rw-rw-r-- 1 hadoop hadoop 3.1K Aug 14 10:29 wordcount.jar
hadoop@sil-shah-VMware-Virtual-Platform:~$
```

Step 6: Run the MapReduce Job

```
Terminal sil-shah-VMware-Virtual-Platform:~$ jps
2036 NameNode
4426 SecondaryNameNode
5186 DataNode
hadoop@sil-shah-VMware-Virtual-Platform:~$ hadoop jar wordcount.jar WordCount /wordcount/input /wordcount/output
2026-08-14 10:33:26,313 ERROR conf.Configuration: error parsing conf mapred-site.xml
com.ctc.wstx.exc.WstxParsingException: Illegal processing instruction target ("xml"); xml (case insensitive) is reserved by the specs.
at [row,col,system-id]: [21,21,"file:/usr/local/hadoop/etc/hadoop/mapred-site.xml"]
    at com.ctc.wstx.sr.StreamScanner.constructWfcException(StreamScanner.java:634)
    at com.ctc.wstx.sr.StreamScanner.throwParseError(StreamScanner.java:504)
    at com.ctc.wstx.sr.BasicStreamReader.readPIPrimary(BasicStreamReader.java:4004)
    at com.ctc.wstx.sr.BasicStreamReader.nextFromProlog(BasicStreamReader.java:2138)
    at com.ctc.wstx.sr.BasicStreamReader.closeContentTree(BasicStreamReader.java:2980)
    at com.ctc.wstx.sr.BasicStreamReader.nextFromTree(BasicStreamReader.java:2728)
    at com.ctc.wstx.sr.BasicStreamReader.next(BasicStreamReader.java:1121)
    at org.apache.hadoop.conf.Configuration$Parser.parseNext(Configuration.java:3405)
    at org.apache.hadoop.conf.Configuration$Parser.parse(Configuration.java:3191)
    at org.apache.hadoop.conf.Configuration.loadResource(Configuration.java:3084)
    at org.apache.hadoop.conf.Configuration.loadResources(Configuration.java:3045)
    at org.apache.hadoop.conf.Configuration.loadProps(Configuration.java:2923)
    at org.apache.hadoop.conf.Configuration.getProps(Configuration.java:2905)
    at org.apache.hadoop.conf.Configuration.<init>(Configuration.java:846)
    at org.apache.hadoop.mapred.JobConf.<init>(JobConf.java:458)
    at org.apache.hadoop.mapreduce.Job.getInstance(Job.java:194)
    at org.apache.hadoop.mapreduce.Job.getInstance(Job.java:214)
    at WordCount.main(WordCount.java:54)
    at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
    at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:62)
    at java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
    at java.base/java.lang.reflect.Method.invoke(Method.java:566)
```

Step 7: View the Generated Word Frequency Output

```

hadoop@sil-shah-VMware-Virtual-Platform:~$ hdfs dfs -ls /wordcount/output
Found 2 items
-rw-r--r-- 1 hadoop supergroup      0 2026-08-14 10:38 /wordcount/output/_SUCCESS
-rw-r--r-- 1 hadoop supergroup    318 2026-08-14 10:38 /wordcount/output/part-r-00000
hadoop@sil-shah-VMware-Virtual-Platform:~$ hdfs dfs -cat /wordcount/output/part-r-00000
a 1
amounts 1
analytics. 1
analyze 1
and 2
computing. 1
data 6
distributed 1
efficiently. 1
for 1
hadoop 4
helps 1
important 1
information. 1
is 2
large 2
mapreduce 2
of 1
organizations 1
process 1
processes 1
processing. 1
provides 1
reliable 1
scalable 1
technologies 1
technology. 1
to 1
used 1
using 1
hadoop@sil-shah-VMware-Virtual-Platform:~$ hdfs dfs -cat /wordcount/output/part-r-00000 > result.txt
hadoop@sil-shah-VMware-Virtual-Platform:~$ cat result.txt
a 1
amounts 1
analytics. 1

```

Step 8: Find Top 10 Most Frequently Occurring Words

```

hadoop@sil-shah-VMware-Virtual-Platform:~$ hdfs dfs -cat /wordcount/output/part-r-00000 | sort -k2 -nr | head -10
data 6
hadoop 4
big 4
mapreduce 2
large 2
is 2
and 2
using 1
used 1
to 1
hadoop@sil-shah-VMware-Virtual-Platform:~$ hdfs dfs -cat /wordcount/output/part-r-00000 | sort -k2 -nr | head -10 > top10.txt
hadoop@sil-shah-VMware-Virtual-Platform:~$ cat top10.txt
data 6
hadoop 4
big 4
mapreduce 2
large 2
is 2
and 2
using 1
used 1
to 1

```

Result: The word-frequency output was sorted in descending order of frequency, and the top 10 most frequently occurring words were identified.

Step 9: Exclude Stop Words

```
hadoop@sil-shah-VMware-Virtual-Platform:~$ hdfs dfs -cat /wordcount/output/part-r-00000
amounts 1
analytics. 1
analyze 1
a CDROM 1
are 1
big 4
computing. 1
data 6
distributed 1
efficiently. 1
for 1
hadoop 4
helps 1
important 1
information. 1
large 2
mapreduce 2
of 1
organizations 1
process 1
processes 1
processing. 1
provides 1
reliable 1
scalable 1
technologies 1
technology. 1
to 1
used 1
using 1
```

Result: Stop words such as a, an, the, and is were excluded from the Mapper, and the MapReduce job successfully generated the updated word-frequency results.

Step 10: Compare Word Frequency for Multiple Input Files

```
hadoop@sil-shah-VMware-Virtual-Platform:~$ hdfs dfs -cat /wordcount/output2/part-r-00000
analytics      1
analytics.     1
and            1
are            1
better         1
big            3
businesses     1
changing       1
data           4
decisions.     1
efficiently.   1
hadoop         2
helps          1
in             1
make           1
processes      1
used           1
widely         1
world.         1
hadoop@sil-shah-VMware-Virtual-Platform:~$ echo "===== INPUT 1 ====="
hdfs dfs -cat /wordcount/output/part-r-00000

echo "===== INPUT 2 ====="
hdfs dfs -cat /wordcount/output2/part-r-00000
===== INPUT 1 =====
amounts 1
analytics.      1
analyze 1
and            2
are            1
big            4
computing.     1
data           6
distributed     1
efficiently.   1
for            1
hadoop         4
helps          1
important      1
information.   1
large          2
mapreduce      2
of             1
organizations  1
process        1
processes      1
```

```

process 1
processes      1
processing.    1
provides       1
reliable       1
scalable       1
technologies   1
technology.    1
to             1
used           1
using          1
INPUT 2 =====
Trash          1
analytics.     1
and            1
are            1
better         1
big            3
businesses     1
changing       1
data           4
decisions.     1
efficiently.   1
hadoop         2
helps          1
in             1
make           1
processes      1
used           1
widely         1
world.         1
hadoop@sil-shah-VMware-Virtual-Platform:~$

```

Result: Word frequency was successfully calculated for two different input files. The frequency of common words varied according to the content of each dataset.

Step 11: Performance Analysis for Different Dataset Sizes

```

hadoop@sil-shah-VMware-Virtual-Platform:~$ ls -lh small.txt medium.txt large.txt
-rw-rw-r-- 1 hadoop hadoop 328K Aug 14 10:49 large.txt
-rw-rw-r-- 1 hadoop hadoop 33K Aug 14 10:48 medium.txt
-rw-rw-r-- 1 hadoop hadoop 335 Aug 14 10:48 small.txt
hadoop@sil-shah-VMware-Virtual-Platform:~$ time hadoop jar wordcount.jar WordCount /wordcount/performance/small /wordcount/performance/out_small
2026-08-14 10:53:27,785 INFO impl.MetricsConfig: Loaded properties from hadoop-metrics2.properties
2026-08-14 10:53:27,875 INFO impl.MetricsSystemImpl: Scheduled Metric snapshot period at 10 second(s).

```

Dataset	Execution Time
Small	3.363 s
Medium	3.361 s
Large	4.364 s

Observation

The execution time for the small and medium datasets is almost the same, while the large dataset takes slightly more time.

This is normal because Hadoop has startup and job-overhead costs. With very small datasets, that overhead can dominate the actual processing time. As the dataset becomes larger, the processing time starts increasing.

Step 12: Final Result, Analysis**12.1 Final Result**

The Word Count application was successfully implemented using Hadoop MapReduce. The text dataset was uploaded to HDFS, processed by the Mapper and Reducer, and the final word-frequency results were stored in HDFS.

The MapReduce execution was completed successfully, and the top 10 frequently occurring words were identified.

12.2 Performance Analysis

The program was tested with datasets of different sizes.

Dataset	Execution Time
Small	3.363 sec
Medium	3.361 sec
Large	4.364 sec

Observation

The large dataset required more execution time than the small and medium datasets. The small and medium datasets had almost identical execution times because Hadoop has some fixed overhead for starting and executing a MapReduce job.

12.3 Key Questions / Analysis**Q1. What is the role of Mapper and Reducer in the Word Count application?****Answer:**

The Mapper reads the input data and generates intermediate key-value pairs in the form (word, 1). The Reducer receives the grouped words and adds their values to calculate the total frequency of each word.

Q2. How are intermediate key-value pairs generated and processed?**Answer:**

The Mapper splits each line of text into individual words and generates (word, 1) for every word. Hadoop then performs Shuffle and Sort, which groups identical words together. The Reducer processes these groups and calculates their total frequency.

Example:

Mapper:

hello → 1

hello → 1

data → 1

Shuffle:

hello → [1,1]

data → [1]

Reducer:

hello → 2

data → 1

Q3. How does HDFS store and distribute the input dataset?

Answer:

HDFS divides large files into blocks and stores these blocks across different DataNodes. It also maintains replicas of the blocks to provide fault tolerance and reliable data storage.

Q4. What advantages does MapReduce offer for processing large datasets?

Answer:

- Processes large amounts of data efficiently.
- Supports parallel processing.
- Provides fault tolerance.
- Works with distributed storage such as HDFS.
- Can scale across multiple machines.
- Automatically handles tasks such as data distribution and grouping.

Q5. How does the execution time vary with dataset size?

Answer:

Generally, execution time increases as the dataset size increases. In our experiment, the large dataset took 4.364 seconds, compared with 3.363 seconds for the small dataset. The small difference between small and medium datasets is due to Hadoop's startup and job-processing overhead.

12.4 Conclusion

Conclusion:

The Word Count application was successfully implemented using Hadoop MapReduce. The Mapper generated intermediate (word, 1) pairs, while the Reducer calculated the total frequency of each word. The input and output data were stored in HDFS. The experiment also demonstrated the effect of dataset size on MapReduce execution time.